Bilkent University
CS491 Senior Design Project

Project Specification Report

UppGrad

T2528

Ali Özhavala (22203113),
Begüm Filiz Öz (22203470),
Bertan Turgut (22203226),
Emre Uçar (22203675),
Koray Sevil (22202548)

Supervisor: Selim Aksoy
November 28 2025

CONTENTS

1. Introduction.....3
1.1 Description.....3
1.2 High Level System Architecture & Components of Proposed Solution.....4
1.2.1 Client Layer.....4
1.2.2 Backend Layer.....4
● User & Profile Service.....5
● Document Service.....5
● Opportunities Service.....5
● Matching Service.....5
● Application & Deadline Tracker.....5
● Notification Service.....5
● AI & Agentic Subsystem.....5
1.2.3 Data Layer.....5
● Relational Database.....6
● Object Storage.....6
● Vector Database.....6
1.3 Constraints.....6
1.3.1. Implementation Constraints.....6
1.3.2. Economic Constraints.....7
1.3.3. Ethical Constraints.....7
1.4 Professional and Ethical Issues.....7
1.5 Standards.....8
2. Design Requirements.....9
2.1. Functional Requirements.....9
2.2. Non-Functional Requirements.....11
2.2.1. Usability.....11
2.2.2. Reliability.....11
2.2.3. Performance.....12
2.2.4. Supportability.....12
2.2.5. Scalability.....13
3. Feasibility Discussions.....14
3.1. Market & Competitive Analysis.....14
3.2. Academic Analysis.....15
4. Glossary.....16
5. References.....17

1. Introduction

1.1 Description

UppGrad is a web application designed to support final-year university students and recent graduates in managing the messy transition from university to early professional and academic life. Instead of letting students manage several job applications, university pages, scholarship sites, and documents all by themselves, UppGrad brings these workflows into a single, personalised environment. Users create a profile describing their academic background, skills, interests, target countries, and career or graduate study goals; this profile then receives recommendations and automations tailored to them.

UppGrad offers a variety of services: job and internship matchmaking based on the student's profile and preferences; graduate program matchmaking for MSc and PhD opportunities; and a scholarship and funding finder to help students with financial needs. Through the document feedback component, students can upload CVs, motivation letters, and statements of purpose, and receive constructive, AI-driven feedback as well as drafted versions for specific roles or programs. A deadline tracker and application manager maintain ongoing applications by students and opportunities that might interest them, sending reminders before important dates and preventing users from missing deadlines. Moreover, a built-in chatbot can answer general questions, offer relocation tips, and provide guidance on exams, visas, and applications.

From a technical perspective, UppGrad is structured around frontend and backend services, and an AI/agentic layer. The frontend manages tasks such as logging in, setting up the user profile, showing dashboards, uploading documents, and a chat interface. On the other hand, the backend implements the main domain logic, such as user & profile management, collecting and storing opportunities (jobs, programs, scholarships), matching and ranking them, document storage, and application/deadline tracking. Finally, an agentic AI orchestrator coordinates specialised agents for tasks like opportunity discovery, profile-aware matching, document review and generation, and (simulated or real) application submission flows. This agent graph allows UppGrad not only to recommend opportunities, but also to plan and execute multi-step workflows. For instance, finding suitable internships, adapting the user's CV and motivation letter, and preparing application bundles can all be executed via the help of AI, turning UppGrad from a static platform into an active assistant for graduate life planning.

1.2 High Level System Architecture & Components of Proposed Solution

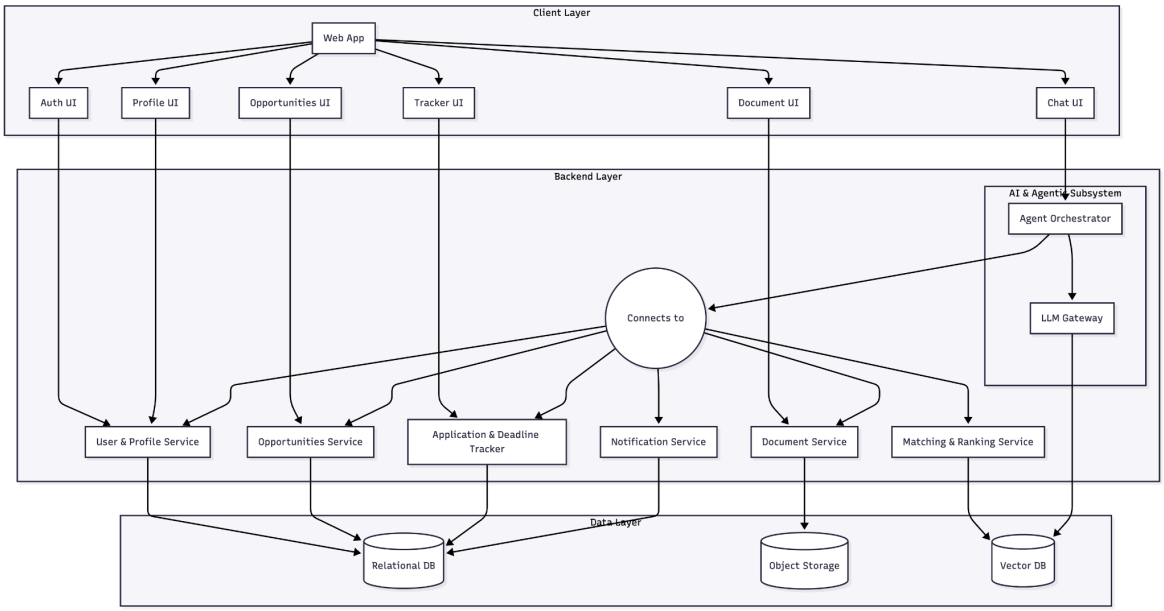

Figure 1. The High Level Architecture of the App

1.2.1 Client Layer

The Client Layer is the primary interface through which users interact with the UppGrad platform. It includes all presentation, input, and interaction components responsible for delivering a consistent, responsive, and user-centered experience. Implemented as a web application, this layer manages rendering logic, front-end state transitions, and communication with backend services through well-defined interfaces. Also it guides users through onboarding, profile configuration, document uploads, and application workflows.

1.2.2 Backend Layer

The Backend Layer forms the operational core of the UppGrad system. It contains the system's domain logic, workflow management, data coordination mechanisms, and intelligent automation features. Its modular organization supports extensibility, maintainability, and separation of concerns. Even though the clear separation between features or domains seem fit for a microservice architecture, we will implement these features in a monolith to avoid the overhead of setting up and configuring individual microservices.

- User & Profile Service

  Manages user accounts, identity-related information, academic history, skills, interests, and preference data. It serves as a central context provider for all services that require personalized processing or recommendation logic.

- Document Service

  Manages user resumes and statement of purposes. Provides AI prepared feedback to these documents.

- Opportunities Service

  Aggregates and maintains structured datasets for jobs, internships, graduate programs, and scholarship opportunities. Provides unified access to external data sources and normalized opportunity records.

- Matching Service

  Implements logic to assess compatibility between users and opportunities. Utilizes traditional algorithms and AI embeddings to generate relevance scores and support personalized recommendations.

- Application & Deadline Tracker

  Manages application records, timelines, and status updates. Supports centralized tracking of deadlines and upcoming tasks across jobs, programs, and scholarships. The job specific timelines will be provided by the user.

- Notification Service

  Generates and dispatches platform notifications, including reminders, alerts, and updates related to deadlines, application changes, or new matched opportunities.

- AI & Agentic Subsystem

  Provides intelligent automation capabilities. In here we will handle AI operations for opportunity discovery, document feedback, and application builder.

1.2.3 Data Layer

In the data layer we will hold records of users, users' tracked applications, user resumes, statement of purpose, and opportunities.

- Relational Database

  Stores structured records including user profiles, opportunity metadata, application histories, deadlines, and notification logs.

- Object Storage

  Manages unstructured content such as CVs and Statements of Purpose. Provides secure and scalable binary file storage with lifecycle management and integration into document-processing workflows.

- Vector Database

  Stores and retrieves high-dimensional embeddings representing documents, user profiles, and opportunity descriptions. Supports efficient similarity search operations for our resume-opportunity matching system.

1.3 Constraints

1.3.1. Implementation Constraints

- UppGrad will be developed as a web application.
- Frontend will be developed using React.
- Backend will be developed using the Django framework of Python.
- We will use OpenAI's API or Gemini API for the AI-related tasks in the project.
- Web scraping and external APIs such as Indeed, Glassdoor, and university portals will be used to collect data such as job listings, PhD positions, etc.
- Git and GitHub will be used for version control and tracking the project.
- A relational or NoSQL database (PostgreSQL/MongoDB) will be used to store user data, CV files, and application history.
- The chatbot will be trained using language models such as GPT.
- AI model training and hosting may require the utilization of cloud platforms such as Google Cloud.
- For deployment, we will use cloud hosting services such as AWS or Google Cloud.

1.3.2. Economic Constraints

- UppGrad will be free to use for newly graduating university students. Ads may be implemented later on to compensate for the costs of hosting.
- APIs to obtain job postings may be costly/expensive, so web scraping is a better option for that. Indeed and/or Glassdoor API are costly (Indeed API may be up to 3 USD per API call)
- Free-tier services from AWS/Google Cloud will be utilized initially, unless we require more than what is available.
- We will use a free CV, motivation letter, statement of purpose etc. datasets for fine-tuning the AI model if we need to.
- The service of GitHub we will use for version control and issue tracking is free.
- PostgreSQL/MongoDB, which we will use for data storage are both free.
- For the AI model, we will use OpenAI's API or Gemini's API. There are free options for Gemini API, so it might be more feasible to use.
- Domain registration and hosting may involve some costs, but we may use GitHub's free domain services for that.
- Free frameworks and open source projects will be used for the development of the project.

1.3.3. Ethical Constraints

- Uploaded CVs, personal data, educational history, and job preferences will be kept confidential and never shared with third parties without consent.
- Multi factor authentication will be provided for the users. The platform will ensure compliance with data privacy regulations for handling personal data.
- Algorithmic bias against gender, ethnicity, school ranking, or location will be eliminated when job recommendations are carried out.
- The career advice chatbot will avoid giving harmful or discriminatory advice.
- If monetization services are implemented in the future, transactions will be carried out safely.
- ACM and IEEE Software Engineering Code of Ethics will be followed during development.
- We have access to a lot of personal data from the users. These include identifiable information such as gender, and also their CVs and motivation letters. We need to protect this data as to not share it with third parties, and ensure that the work of the users are protected. For example, these CVs or letters should not be further used to train the AI models if the users are not comfortable with that.

For services such as that, opt-in and opt-out options will be provided for data analytics.
- The user will be informed about data handling practices in a transparent manner via a privacy policy.
- Hashing and authentication will be used to protect the users' login info.
- The conversations the user will have with the chatbot will be confidential.
- The agentic AI system to apply to jobs has to somehow be a human-in-the-loop system to ensure that we are not applying to jobs the user is not interested in.
- Accessibility and inclusion principles in UI/UX (WCAG principles) will be followed.

1.4 Professional and Ethical Issues

- Indeed and Glassdoor API's are hard to obtain, especially since we are not a cooperation or partner to them. So getting these APIs might not be feasible, hence we will probably need to carry out web scraping instead.
- As for teamwork, we will abide by the rules of academic integrity and work with our own code.
- As a team, everyone will try to contribute equally.
- We will maintain transparency in the application, if any data is used, we will inform the users accordingly.
- Code reviews will be done in collaboration to give proper feedback and ensure a smooth development timeline.
- Documentation and communication is important to ensure that team members are working in harmony with each other and to prevent future conflicts.

1.5 Standards

- OWASP security guidelines will be followed to prevent vulnerabilities.
- IEEE and ACM Software Engineering Code of Ethics will be followed, as mentioned previously.
- We will follow recommended IEEE standards and practices for software design.
- GDPR principles will be considered for user privacy.
- WCAG accessibility standards will be considered to provide accessibility for all users.
- REST API standards will be followed.
- ISO/IEC 27001 will be followed to securely handle user data and stored documents.
- ISO/IEC 25010 will be followed for enabling software quality aspects (usability, reliability, and maintainability, etc.).

2. Design Requirements

2.1. Functional Requirements

- The system lets users create an account with an email address and password, log in and log out, checks that the email address is valid and unique, sends a verification email after sign up, and provides a password reset flow when users forget their password.

- Users can delete their own accounts, and when they do, personal information and uploaded documents are removed from active storage, while anything kept for statistics is stored only in anonymised form.

- After the first login, the system guides users through a short profile flow that collects education, skills, interest areas, preferred countries or cities, and the types of opportunities they are looking for, and users can edit this profile at any time.

- Users can define simple goals such as finding a certain kind of role within a time frame, and can choose whether their anonymised data is allowed to be used for internal analytics and future improvements of recommendations and feedback.

- Users can upload CVs and different kinds of letters in common formats such as PDF or DOCX, see all of their files in a list, open them, download them, and delete any document they no longer want to keep.

- For each uploaded document, the system extracts structured information such as education entries, work experience, and listed skills, and also stores a vector representation so that this information can be reused in matching and feedback.

- Users can request general feedback on any CV or letter, and the system returns a structured response that points out strengths, weaknesses, missing sections, and practical suggestions for improvement in a format ready to show in the interface.

- When a user selects an opportunity and one of their documents together, the system compares the opportunity requirements with the document content and highlights which points are covered well and which areas appear weak or missing.

- For a chosen opportunity, the system can generate an initial motivation letter or statement draft using templates and the data in the user profile, and it can also produce revised versions of an existing letter so that it matches that opportunity more closely.

- The system collects opportunities from allowed external sources such as public job boards or university pages, and normalises basic fields such as title, institution, location, deadline, short requirement summary, and external application link.

- Users can search for opportunities with free text and refine the results with filters such as country, field, level, work modality, and deadline range, and the search results show enough summary information to decide whether to open the full details.

- A matching component calculates relevance scores between user profiles and available opportunities using profile fields, simple rules, and vector similarity, and the dashboard shows a ranked list of recommended jobs, internships, programs, and scholarships for each user.

- Users can save opportunities that interest them and remove them from the saved list, and each opportunity has a detail view that shows the full description, key requirements, deadline if it is known, and the external application link.

- For any saved opportunity, users can create an application record, choose which CV and letter they will use, set and update the application status such as planned, submitted, interview, rejected, or accepted, and see these records as their application history, including deadlines and small task lists with checkboxes for things like references or transcripts.

- The system sends reminders for approaching deadlines and task due dates inside the site and by email, lets users turn on alerts based on saved filters or goals when new matching opportunities arrive, provides a chat interface for questions about career planning and specific postings, guides users step by step through preparing an application for a chosen opportunity, and offers an internal admin view where authorised staff can see basic statistics and adjust source related settings.

2.2. Non-Functional Requirements

2.2.1. Usability

- **Simple and consistent interface**
  Main features such as profile editing, opportunity search, document upload, and application tracking shall be reachable in a small number of clicks from a clear navigation bar. Labels and icons shall use consistent wording.

- **Low learning curve and feedback on errors**
  A new user familiar with common web sites shall be able to use core features without reading a manual. Forms shall show immediate validation messages for missing or invalid input and shall explain how to correct it.

- **Accessibility and language readiness**
  The interface shall follow basic accessibility practices such as sufficient contrast, keyboard navigation, and descriptive labels for assistive technologies. Text strings shall be centralised so that additional languages can be added without code changes.

2.2.2. Reliability

- **Service availability and graceful failure**
  The system should be available during typical student working hours, with only short planned maintenance periods. If external sources or AI services fail, core features such as viewing profiles, saved opportunities, and stored documents shall remain available, and the system shall clearly indicate which functions are temporarily limited.

- **Monitoring and logging**
  Backend errors, failed external calls, and unexpected exceptions shall be logged with enough context to debug issues. Serious failures should generate alerts for the development team.

2.2.3. Performance

- **Response times for common actions**

  Under normal network conditions, the dashboard, search results, and application pages should open within a few seconds. Small changes such as editing profile information or updating an application status should also be confirmed within a similar time, so the system feels responsive rather than slow or stuck.

- **Document processing and user feedback**

  AI based feedback on CVs and letters can take longer than a simple page load, since it requires heavier processing. While this is running, the interface shows clear progress, and if it takes more time, the user can continue browsing other parts of the site while the system finishes the analysis in the background.

- **Efficient queries and refreshed recommendations**

  The data model and indexes are set up so that typical queries for opportunity lists, saved items, and notifications remain fast even as the number of users and postings grows. When a user changes important profile fields, the recommendations on the dashboard are refreshed within a short and predictable time, instead of lagging behind the new preferences.

2.2.4. Supportability

- **Configuration and environment separation**
  Secrets and environment specific settings such as database addresses and API keys shall be stored in configuration or environment variables, not hard coded in the source.

- **Documentation and testing**
  The project shall include basic technical documentation for architecture, data models, and deployment steps. Core backend logic, especially data access and matching, shall have at least basic automated tests to detect regressions.

2.2.5. Scalability

- **Handling growth in users and data**
  The chosen relational and vector databases shall support growth in user count and opportunity volume without severe performance loss. Indexing and partitioning strategies shall be planned with expansion in mind.

- **Separation of compute intensive tasks**
  Components that require heavier processing, such as AI feedback and scraping jobs, shall be deployable as separate workers or services so that they can scale independently of the main web application.

3. Feasibility Discussions

3.1. Market & Competitive Analysis

UppGrad aims to offer fresh graduates fast and practical tools for their careers. They may struggle with finding relevant job and internship offers that are suitable for their profile, tracking multiple deadlines simultaneously, searching for suitable scholarships and graduate programs, and preparing good CVs and motivation letters. Existing platforms in the market tend to focus on one aspect of this process, e.g. showing relevant position offers or CV checking. Although they offer practical solutions, students should still use more than one such helper application to ease their work. UppGrad will fill this gap by incorporating multiple powerful AI tools ready for students' use and all inside the same application.

WayUp and Handshake provide access to large job and internship databases, but instead of matching users via profile analysis, they rely mostly on tag-based matching, which reduces personalisation [1][2]. These platforms are practical in terms of opportunity discovery, but they lack other tools that UppGrad will offer, e.g. document preparation or intelligent career planning.

VMock and Jobscan focus mainly on CV improvement by employing distinct scoring mechanisms [3][4]. Although they are useful for acquiring basic résumé feedback based on pre-determined rules, they may fall short on providing context-aware feedback and dynamically tailoring recommendations to the specific requirements of a job posting or a program.

Lastly, platforms such as simplify.jobs and loopcv aim to reduce the manual workload of application processes by automating repetitive form-filling tasks [5][6]. However, they may lack intelligent adaptation or editing capabilities for distinct applications and thus may overlook specific requirements and expectations of employers or academic institutions.

In conclusion, UppGrad is designed to unify job or academia-related opportunity discovery, intelligent and adaptive document analysis, and deadline tracking in a single website. This way, it will be a logical option to try among other applications in the market for freshly graduated students.

3.2. Academic Analysis

Recent academic research supports the use of artificial intelligence systems to help students with career planning and educational decisions, but current systems still have clear limitations. Bahalkar et al. show that AI-based predictive models can generate personalised career and subject recommendations by analysing students' academic performance, personal interests, and background information [7]. However, their approach mainly relies on structured academic datasets, which may not fully represent real-world student behaviour and decision-making. Although their results show good prediction performance, it is still unclear how well such models can scale and work across different countries, institutions, and education systems.

Similarly, Nalawade and Tiple show that machine learning–based university recommendation systems can help students choose suitable graduate programs using test scores, academic history, and past admission data [8]. However, their system is built mainly on historical records and fixed rules, and this may introduce bias. Besides, it may also face some difficulties when admission policies change or when students come from less typical educational backgrounds. Their work demonstrates that AI-based recommendation systems are possible, but it also highlights that they need to be more flexible and adaptable.

These two studies suggest that UppGrad is realistic to implement, academically meaningful, and technically feasible as a software engineering project, but careful design is needed to handle real-world challenges such as fairness and adaptability.

4. Glossary

CV = Curriculum Vitae

AI = Artificial Intelligence

UI = User Interface

UX = User Experience

API = Application Programming Interface

OWASP = Open Worldwide Application Security Project

IEEE = Institute of Electrical and Electronics Engineers

ACM = Association for Computing Machinery

GDPR = General Data Protection Regulation

WCAG = Web Content Accessibility Guidelines

REST API = Representational State Transfer Application Programming Interface

ISO/IEC 27001 = Information Security Management System Standard by the International Organization for Standardization and International Electrotechnical Commission

ISO/IEC 25010 = System and Software Quality Requirements and Evaluation — System and Software Quality Models

5. References

[1] “Wayup,” Jobs & Internships for College Students and Recent Grads, https://www.wayup.com/ (accessed Nov. 27, 2025).

[2] “Your career starts here,” Handshake, https://joinhandshake.co.uk/ (accessed Nov. 27, 2025).

[3] “Career acceleration platform,” VMock, https://www.vmock.com/ (accessed Nov. 27, 2025).

[4] Optimize your resume and boost interview chances - jobscan, https://www.jobscan.co/ (accessed Nov. 27, 2025).

[5] Simplify | your entire job search in one place., https://simplify.jobs/ (accessed Nov. 27, 2025).

[6] “The Job Search Automation Platform,” Loopcv, https://www.loopcv.pro/tr/ (accessed Nov. 27, 2025).

[7] P. Bahalkar, P. Peddi, and S. Jain, “AI-Driven Career Guidance System: A Predictive Model for Student Subject Recommendations Based on Academic Performance and Aspirations,” Frontiers in Health Informatics, vol. 13, no. 3, pp. 8216–8230, 2024.

[8] A. Nalawade* and B. Tiple, “The university recommendation system for Higher Education,” International Journal of Recent Technology and Engineering (IJRTE), vol. 8, no. 6, pp. 1692–1696, Mar. 2020. doi:10.35940/ijrte.f7632.038620